

AA-04

API Contract Reference

Every admin, partner and fleet endpoint at freeze, with scope and idempotency

30 August 2026 · Rev 1.0 — baseline

ENDPOINTS AT FREEZE

164

ADMIN NAMESPACE

93

PARTNER NAMESPACE

34

FLEET NAMESPACE

37

IDEMPOTENCY-KEYED

70

STEP-UP GUARDED

24

This is the contract, not a description of it. Every path below exists in packages/shared/routes.ts, is declared with the permission printed beside it, and is exercised by the contract sweep in AA-01 section 1.7. At the end of week two these paths and their request and response shapes freeze: after that a change needs a version bump of @atavishaala/contract and a line in the Monday sync. Read AA-02 first for the middleware chain every one of these runs through, and AA-08 for the permissions.

4.1 HOW TO READ THIS DOCUMENT

The freeze rule, and the four conventions every endpoint obeys

There are 164 endpoints at freeze: 93 under /api/v1/admin, 34 under /api/v1/partner and 37 under /api/v1/fleet. The mobile and public namespaces are already shipped and are not re-listed here; they appear only in section 4.9 where the additive-only rule that protects them is stated. Each row below carries the HTTP method, the path, the permission string the route declares, whether the route consumes an Idempotency-Key, and one line of note. The permission column is the literal argument to requirePermission; two permissions separated by a plus sign mean the route declares both and both must resolve to ALLOW under the DENY-wins rule of AA-08 section 8.4.

At the end of week two this document freezes. Freeze means the path, the method, the permission, the request shape and the response shape stop moving. It does not mean the handler stops changing. After the freeze a contract change costs a minor version bump of @atavishaala/contract, a regenerated client in every consuming repository, and a line in the Monday sync so the three front-end pairs know before their build breaks. The alternative we rejected was a soft freeze where breaking changes are allowed if the author tells everyone: that failed on the previous programme because "everyone" was a Slack message that two people read on a Friday.

Four conventions hold across every namespace. They are stated once here rather than repeated in 164 note fields, so a row that says nothing about pagination still paginates the way 4.1.1 describes.

4.1.1 Pagination is cursor-based, never offset

Every list endpoint accepts ?cursor= and ?limit=, returns at most 100 rows, and defaults to 25. The cursor is an opaque base64 string; the client must treat it as a token and must never construct one. Inside it is the sort key of the last row returned plus the row id as a tiebreak, which is why every list endpoint has a deterministic total order even when the visible sort column has duplicates.

Offset paging lost for one concrete reason: the admin exception board and the dispatch queue are both sorted by a column that changes while the operator is reading. Under concurrent inserts an offset of 50 shows rows the operator has already seen and silently skips rows they have not, and on an exception board a skipped row is a customer who never gets called back. Cursor paging is stable under insert because the cursor names a position in the sort, not a count of rows before it. The cost we accepted is that there is no "jump to page 7" and no total count on large lists; the count endpoints that do exist are separate and approximate.

4.1.2 Optimistic concurrency by version integer

Every mutable row that two operators can plausibly open at once carries a version integer. A GET returns it. A PATCH or PUT that changes such a row must send it back, and the handler writes with WHERE id = \$1 AND version = \$2, incrementing version in the same statement. If the update affects zero rows the request fails with 409 STALE_WRITE and the response carries the current version so the portal can refetch and show a diff.

This is If-Match semantics carried in the body rather than in an ETag header. The header form is more correct and we did not use it, because two of the three portals post multipart forms for the same resources and threading a header through the upload path was more work than a field. Rows that are append-only do not have a version: you cannot have a stale write against a ledger because you cannot write a ledger row twice. Attempting an UPDATE there is 409 LEDGER_IMMUTABLE from the database rule, not from the handler (AA-03 section 3.9).

4.1.3 Idempotency-Key on every route marked yes

70 of the 164 routes consume an Idempotency-Key header. The rule for which ones is mechanical rather than aesthetic: a route is keyed if replaying it would move money, create a second physical action, or claim a scarce resource. A route that only edits a display field is not keyed, because the second write is harmless and the key would be pure ceremony. Sending a key to a route that does not declare one is ignored, not an error, so a generic client wrapper may attach one everywhere.

The mechanism is in AA-02 section 2.6 and is not repeated here. What matters at the call site is the three outcomes: a first attempt runs, a replay with the same key and the same body returns the stored response byte-for-byte with Idempotent-Replay: true, and a replay with the same key and a different body is 409

IDEMPOTENCY_MISMATCH. Keys are scoped to (actor_kind, actor_id, endpoint, key) and expire after 24 hours, so a client that reuses one key per user action across two endpoints is safe.

4.1.4 The envelope

Every response, on every path, in every namespace, is one of exactly two shapes. There is no third shape for lists, no bare array, and no 200 with an error inside it. The error code is the contract; the message is for a log and may be reworded without a version bump.

```

SUCCESS 200 / 201
{
  "data": { ... } | [ ... ],
  "cursor": "eyJrIjoiMjAyNi0wOC0zMCI9"
    | null
}

201 also sets: Location: /admin/...
Replays also set: Idempotent-Replay: true
429 also sets: Retry-After: 30

ERROR 4xx / 5xx
{
  "error": {
    "code": "STALE_WRITE",
    "message": "row moved since read",
    "details": [
      { "path": "version", "have": 4, "want": 7 }
    ]
  }
}

```

4.2 AUTHENTICATION FLOWS

Four audiences, three portal login sequences, one step-up

A token is minted for exactly one audience and is rejected everywhere else. The aud claim is checked by `requireActor`, which sits before `requirePermission` in the chain for the reason given in AA-02 section 2.3.2, and a token presented to the wrong namespace fails with 401 ACTOR_MISMATCH before any permission lookup happens. That ordering is what stops a fleet token from producing a permission-denied audit trail against admin resources: the request never gets far enough to have a permission evaluated.

ACTOR	AUDIENCE	NAMESPACE	CREDENTIAL	TOTP
customer	atv_app	/api/v1/app	Mobile OTP (WhatsApp, SMS fallback)	No
staff	atv_admin	/api/v1/admin	Email + Argon2id password + mandatory TOTP	Yes, always
partner	atv_partner	/api/v1/partner	Email + Argon2id password, TOTP for finance	Conditional
fleet	atv_fleet	/api/v1/fleet	Email or mobile + Argon2id, TOTP for finance	Conditional

Access tokens for the three portals live 900 seconds; the mobile app gets 600. Refresh tokens live 7 days for admin, 14 for partner and fleet, 30 for the app. The admin window is the shortest on purpose: an admin refresh token is the single most valuable secret a laptop can leak, and a dispatcher who has to log in once a week is a cost we are willing to charge. Refresh is rotating with family revoke: presenting a refresh token returns a new one and invalidates the old, and presenting an already-used token revokes the entire family and forces a full re-login, because a replayed refresh token means either a broken client or a stolen one and we cannot tell which from inside the handler.

4.2.1 Staff login — /api/v1/admin

1. POST /admin/auth/login with email and password. Argon2id verify against `staff_users`. On success the response is not a session: it is { "challenge_token": "...", "factor": "totp" } with a 300-second life. On failure it is 401 with a constant-time delay and no hint about which half was wrong.
2. POST /admin/auth/totp with the challenge token and the six digits. This is the only place a staff session is created. TOTP is mandatory for every one of the nine roles in `ROLES_ADMIN` including `SUPER_ADMIN`, and a `StaffUser` row with no enrolled factor cannot reach step two at all, which is why enrolment happens at creation time and not at first login.
3. The response sets the access token in the body and the refresh token as an `HttpOnly`, `Secure`, `SameSite=Strict` cookie scoped to the API host. The access token is held in memory by the portal and never in `localStorage`; a reload costs a silent refresh call.
4. GET /admin/me returns the staff row, the resolved permission set after DENY-wins, and the timestamp of the last verified factor. The portal uses that timestamp to grey out step-up actions it knows will fail rather than letting the operator fill a form and lose it.
5. POST /admin/auth/refresh rotates. POST /admin/auth/logout revokes the family and clears the cookie; it is idempotent and a second call is still 204.

4.2.2 Partner login — /api/v1/partner

1. POST /partner/auth/login with email, password and the org slug. The slug is in the body and that is not a violation of I-03: tenancy in a request is forbidden once a token exists, and here no token exists yet. The slug disambiguates a person who works for two supplier orgs with the same email; it is matched against `partner_orgs` and never trusted afterwards, because the `partner_org_id` that every later request is scoped by comes from the minted

claim.

2. POST /partner/auth/totp only when the user has enrolled a factor or holds a finance role. Partner TOTP is conditional rather than mandatory because a supplier packing clerk on a shared warehouse terminal will not carry an authenticator, and forcing one produced a shared secret taped to the monitor when we tried it. Anyone who can touch a bank account or a payout view is in the mandatory set.
3. POST /partner/auth/accept-invite carries a single-use invite token with a 72-hour life and sets the first password. There is no self-registration on this namespace: a partner org exists only because an admin created it and verified it.
4. GET /partner/me returns user, org, role, permissions, org status and KYC completeness. The portal reads KYC completeness to decide whether to show the finance tab at all, since an unverified bank account makes every payout view empty and confusing.

4.2.3 Fleet login and the rider path — /api/v1/fleet

1. POST /fleet/auth/login accepts email and password for office roles. A FLEET_RIDER logs in by mobile instead, because a rider has a phone and no email that they check.
2. POST /fleet/auth/otp/request sends a six-digit code over the same WhatsApp-first channel the mobile app uses, with SMS fallback. The code is stored as a bcrypt hash and nothing else: not in the row, not in a log line, not in a Sentry breadcrumb. This route is idempotency-keyed so that a rider double-tapping in a weak-signal area does not burn two codes and two WhatsApp template sends.
3. POST /fleet/auth/login with the mobile and the code completes the rider session. Failed attempts increment a counter on the login row and lock the mobile for 15 minutes after five, which is deliberately looser than the delivery-OTP lockout in 4.6.3 because a locked-out rider at 7 a.m. cannot start work.
4. POST /fleet/auth/totp covers owner and finance roles only, on the same reasoning as partner.
5. GET /fleet/me returns user, org, role, tier, current COD exposure and the blocked flag. The blocked flag matters at the top of the app: a blocked fleet can still see jobs it holds but every accept will fail, and finding that out one offer at a time is a bad experience.

4.2.4 Step-up: the 300-second fresh factor

24 of the 164 routes are guarded by requireFreshFactor(300). The guard does not ask whether the caller has a TOTP factor; it asks when they last proved it. If the last verified factor is older than 300 seconds the request never reaches the handler and the response is 401 STEP_UP_REQUIRED carrying a challenge object. The portal opens StepUpDialog, the operator types six digits, the portal calls the namespace step-up route, and then replays the original request with the same body and the same Idempotency-Key. Because the key is unchanged and the first attempt never created an idempotency row, the replay is a first attempt, not a replay.

```
POST /admin/partners/f3a1/status --> 401 STEP_UP_REQUIRED
{ "status": "SUSPENDED", "reason": "..."} { "error": { "code": "STEP_UP_REQUIRED",
"message": "fresh factor required",
"challenge": { "factor": "totp",
"max_age_seconds": 300 } } }
Idempotency-Key: 7c2e-...

POST /admin/auth/step-up { "code": "418293"} --> 204, last_factor_at = now()

POST /admin/partners/f3a1/status --> 200 (same body, same Idempotency-Key)
```

WHY 300 SECONDS, NOT PER-ACTION RE-AUTH

A dispatcher suspending four partners in one sitting types six digits once, not four times. Per-action re-auth was rejected for exactly that reason: the observed workaround is an authenticator app left open next to the keyboard, which is worse than a five-minute window. 300 seconds is also short enough that an unlocked laptop in a shared office is not a standing grant.

A COUNT THAT HAS DRIFTED

I-09 in AA-00 says eleven step-up endpoints. Counted from this contract the number is 24, because the enumeration grew after the invariant was written. The contract is the source of truth and the invariant text needs a patch in aa_facts before the freeze; the rule itself, requireFreshFactor(300), has not changed.

4.3 ADMIN NAMESPACE ENDPOINTS

93 routes in ten groups under /api/v1/admin

The admin namespace is the largest of the three at 93 routes, and it is the only one where the caller is an Atavishaala employee rather than a counterparty. That changes the threat model rather than removing it: a hostile party cannot register for this namespace, but a support agent with the wrong role can still read a customer they have no business reading, which is why every read here is permission-declared and every masked field stays masked regardless of role (I-06). The nine roles in ROLES_ADMIN are compositions of the 56 permission rows in AA-08 section 8.2; the strings in the PERMISSION column below are those rows, not role names.

Read the ten groups as the ten screens families of AA-05. Where a route exists that no screen calls it is marked in the note, and where a screen needs two calls the second is usually a count or a lookup that is deliberately not joined into the first.

4.3.1 Session — seven routes

Login, factor, refresh, logout, enrolment, step-up and identity. The one thing to get right is that /admin/auth/step-up is not a login: it mints nothing, it stamps last_factor_at on the session already in hand, and requireFreshFactor(300) reads that stamp. Treating it as a login produces a portal that throws away the operator's half-filled form and then asks them to start again.

METHOD	PATH	PERMISSION	IDEM	NOTE
POST	/admin/auth/login	public	-	Email + password. Returns a TOTP challenge, not a session.
POST	/admin/auth/totp	challenge token	-	Completes login. Sets the atv_admin refresh cookie.
POST	/admin/auth/refresh	refresh cookie	-	Rotates the family. Reuse revokes the whole family.
POST	/admin/auth/logout	session	-	Revokes the current family only.
POST	/admin/auth/totp/enrol	session	-	Returns otpauth URI and ten recovery codes, once.
POST	/admin/auth/step-up	session	-	Fresh TOTP for a step-up action. Valid 300 s.
GET	/admin/me	session	-	Profile plus the flattened effective permission array.

4.3.2 Catalog — eleven routes

Products, variants, prices, stock, media, categories and the moderation inbox. The trap is POST /admin/products: it creates house products only. A partner-authored product never enters through this route; it arrives as a DRAFT from /partner/products and leaves through /admin/products/:id/moderate. Two writers into the same table under different rules is exactly how a product reaches the app without ever passing moderation, so the handler rejects a body carrying partner_org_id with 400 VALIDATION_FAILED rather than quietly accepting it.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/admin/products	catalog.read	-	Filter by partner, moderation state, category, stock band.
POST	/admin/products	catalog.write	-	House products only. Partner products arrive via review.
GET	/admin/products/:id	catalog.read	-	Includes variant list, media, moderation history.
PATCH	/admin/products/:id	catalog.write	-	Partial update. Diff goes to audit_log.
POST	/admin/products/:id/moderate	catalog.moderate	-	body: APPROVE REJECT DELIST + reason.
PATCH	/admin/variants/:id/price	catalog.price.override	-	Step-up. Writes a pricing_snapshot.
POST	/admin/variants/:id/stock	catalog.stock.adjust	yes	Signed delta + reason code.
DELETE	/admin/media/:publicId	catalog.media.delete	-	Deletes in Cloudinary and in product_media.
GET	/admin/moderation/queue	catalog.moderate	-	The review inbox, oldest first, with SLA age.
GET	/admin/categories	catalog.read	-	Full tree, cached 60 s at the edge.

METHOD	PATH	PERMISSION	IDEM	NOTE
PUT	/admin/categories/:id	catalog.write	-	Rename, reparent, reorder.

4.3.3 Orders — ten routes

Orders, their shipments, the ops board and the return queue. There is no route that writes orders.status and there will not be one: the value is derived by `reduceOrderStatus()` from the shipment set on every read (I-11), and the 28 combinations are enumerated in AA-03 section 3.7. The only way to move an order is to move a shipment, and the only way to move a shipment against the FSM is POST `/admin/shipments/:id/transition`, which is step-up guarded and demands a reason of at least 20 characters because that reason is the only evidence anyone will have six weeks later.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/admin/orders	order.read	-	Server-side pagination, saved-view compatible.
GET	/admin/orders/:id	order.read	-	Order, shipments, items, payments, events, masked customer.
GET	/admin/orders/:id/events	order.read	-	The append-only timeline.
POST	/admin/orders/:id/cancel	order.cancel	yes	Releases stock, reverses payment or COD.
POST	/admin/orders/:id/notes	order.note.write	-	Internal note. Never shown to the customer.
GET	/admin/shipments	order.read	-	The ops board. Groups by state and ready_by breach.
POST	/admin/shipments/:id/resplit	order.shipment.reassign	yes	Moves items to a new shipment.
POST	/admin/shipments/:id/transition	order.transition.force	yes	Step-up. Requires a reason $\geq$ 20 chars.
GET	/admin/returns	order.return.review	-	Return queue with photo evidence thumbnails.
POST	/admin/returns/:id/decide	order.return.review	yes	APPROVE REJECT + reason + refund mode.

4.3.4 Delivery — eight routes

The dispatch side: jobs, the exception board, direct assignment, rebroadcast, cancellation and the POD review flag. Assign and rebroadcast look similar and are not: assign skips the auction and binds the job to one fleet immediately, while rebroadcast puts it back into the wave machinery with the wave chosen by the operator. Rebroadcasting a job a fleet has already accepted is a 409, not a silent re-offer, because a rider who is already driving to the supplier gate must not find the job gone.

GET `/admin/jobs/unassigned` is the screen ops actually lives on. It is also the reason section 4.10 is honest about polling: with no server-sent events on this namespace the board refetches on a 20-second timer, and that timer is the single largest source of Cloud Run requests in the programme.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/admin/jobs	job.read	-	Filter by state, fleet, pincodes, promised_by breach.
GET	/admin/jobs/:id	job.read	-	Offers, rider, scans, POD records, geo distance.
GET	/admin/jobs/unassigned	job.read	-	The exception board. The screen ops lives on.
POST	/admin/jobs/:id/assign	job.assign	yes	Direct assignment, skipping the auction.
POST	/admin/jobs/:id/rebroadcast	job.rebroadcast	yes	Choose wave 1, 2, 3 or ALL.
POST	/admin/jobs/:id/cancel	job.cancel	yes	Step-up. Returns the shipment to READY_FOR_PICKUP.
POST	/admin/pod/:id/flag	job.pod.review	-	Marks a POD suspect: geo outside fence or reused photo.
GET	/admin/serviceability/:pincodes	job.read	-	Which fleets cover it, at what fee, COD allowed.

4.3.5 Money — nine routes

Payments, refunds, the customer wallet and coupons. POST `/admin/refunds` does not move money. It creates a refund row in PENDING_APPROVAL and nothing else; the gateway is touched only by `/admin/refunds/:id/approve`, which needs a different permission, a fresh factor, and a staff id different from the initiator. Two humans on

every rupee that leaves is the cheapest control we have, and it cost one extra table column.

Wallet credit and debit are step-up guarded and reason-coded, and a debit cannot take a balance below zero. There is no cash-out route anywhere in this contract, in any namespace: paying wallet balance to a bank account makes the wallet a prepaid payment instrument and needs an RBI licence we do not hold. Wallet value can only be spent on the platform, and that is a product constraint we chose to live with rather than a gap to close later.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/admin/payments	payment.read	-	Razorpay and COD, with settlement state.
GET	/admin/payments/:id	payment.read	-	Gateway reference masked to last four.
POST	/admin/refunds	refund.initiate	yes	Creates a PENDING_APPROVAL refund. Never pays out.
POST	/admin/refunds/:id/approve	refund.approve	yes	Step-up. Chooses GATEWAY or WALLET.
POST	/admin/wallet/:userId/credit	wallet.credit	yes	Step-up. Reason code mandatory.
POST	/admin/wallet/:userId/debit	wallet.debit	yes	Step-up. Cannot take a balance negative.
GET	/admin/wallet/:userId/ledger	payment.read	-	Append-only entries, newest first.
GET	/admin/coupons	coupon.write	-	Active, scheduled and expired.
POST	/admin/coupons	coupon.write	yes	Validated against overlap with live coupons.

4.3.6 Payouts — nine routes

Accrual, runs, withholding, COD reconciliation and the partner rate card. The lifecycle is DRAFT to LOCKED to RELEASED and it is one-way. Lock freezes membership and generates the bank file; release records the UTR and is irreversible, because the money has left the account by the time the request returns. A failed bank transfer is therefore not a retry of release: it is a fresh run for the same period, per RB-06, and the second run picks up only the lines the first did not settle.

PUT /admin/partners/:id/rate-card is effective-dated and never edits in place. The reason is reconstruction: a commission dispute three months from now needs the rate card as it stood on the order date, and an in-place edit destroys that. The same reasoning is why the whole payout chain is idempotency-keyed and why section 4.6.4 spends a page on the run key.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/admin/payouts/accrued	payout.read	-	Group by org and kind. What the next run will contain.
GET	/admin/payout-runs	payout.read	-	History with totals and UTR last four.
POST	/admin/payout-runs	payout.run.create	yes	kind + period. Assembles a DRAFT run.
POST	/admin/payout-runs/:id/lock	payout.run.create	yes	Freezes membership. Generates the bank file.
POST	/admin/payout-runs/:id/release	payout.run.release	yes	Step-up. Records the UTR. Irreversible.
POST	/admin/payout-lines/:id/withhold	payout.line.withhold	yes	Reason ≥ 20 chars. Removed from the run.
GET	/admin/cod	payout.read	-	Open COD exposure by fleet, with age buckets.
POST	/admin/cod/:id/reconcile	cod.reconcile	yes	Step-up. Records received amount and variance.
PUT	/admin/partners/:id/rate-card	commission.write	yes	Step-up. Effective-dated, never in place.

4.3.7 Partners — eight routes

Supplier org lifecycle: invite, KYC review, bank verification, status and user management. PATCH /admin/partners/:id declares two permissions because it spans two kinds of field: partner.read covers display name and contact, partner.status.write covers anything that changes the org's commercial standing. Status itself is not patchable at all and goes through /admin/partners/:id/status with step-up, since a suspension delists every SKU that org has live and is visible to customers within one cache cycle.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/admin/partners	partner.read	-	With status, live SKU count, 30-day GMV, on-time rate.
POST	/admin/partners	partner.invite	yes	Creates a DRAFT org and emails the owner invite.
GET	/admin/partners/:id	partner.read	-	Org, users, KYC docs, rate cards, bank last four.
PATCH	/admin/partners/:id	partner.read + partner.status.write	-	Display name, cycle, contact.
POST	/admin/partners/:id/status	partner.status.write	yes	Step-up. Suspension delists all SKUs.
POST	/admin/partner-kyc/:id/review	partner.kyc.review	yes	VERIFY REJECT + note.
POST	/admin/partner-bank/:id/verify	partner.bank.verify	yes	Step-up. Manual ₹1 check at MVP.
POST	/admin/partner-users/:id/manage	partner.user.manage	-	DISABLE RESET REROLE.

4.3.8 Fleets — nine routes

Delivery-partner org lifecycle, plus tiers, service areas and rider suspension. The route to watch is POST /admin/fleets/:id/tier. It is not step-up guarded and it looks like a settings toggle, but tier decides broadcast wave membership, which decides who is offered work, which decides who earns. Treat it as a commercial change: the audit row it writes will be read in a dispute, and an operator who moves a fleet from tier 1 to tier 3 without a note has left no defensible record.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/admin/fleets	fleet.read	-	With tier, live riders, acceptance rate, on-time rate.
POST	/admin/fleets	fleet.invite	yes	Creates a DRAFT fleet org and emails the owner invite.
GET	/admin/fleets/:id	fleet.read	-	Org, users, riders, vehicles, areas, COD exposure.
POST	/admin/fleets/:id/status	fleet.status.write	yes	Step-up. Suspension withdraws live offers.
POST	/admin/fleets/:id/tier	fleet.tier.write	-	Tier 1..3. Changes broadcast wave membership.
POST	/admin/fleet-kyc/:id/review	fleet.kyc.review	yes	VERIFY REJECT + note.
GET	/admin/fleet-areas/requests	fleet.area.approve	-	Pending service-area and rate changes.
POST	/admin/fleet-areas/:id/approve	fleet.area.approve	yes	APPROVE REJECT + note.
POST	/admin/riders/:id/suspend	rider.suspend	yes	Reason code. Reassigns that rider's live jobs.

4.3.9 Advisory — seven routes

The agronomist queue: claim, notes, reassignment and the SLA override. Claim is atomic and the second claimer gets 409 ALREADY_CLAIMED, which means a portal that optimistically paints the card as claimed before the response arrives has to unpaint it. Reassignment moves the claim but keeps the original clock, because the farmer who asked the question does not get a fresh SLA just because the first agronomist went off shift.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/admin/advisory/queue	advisory.read	-	Ordered by sla_due_at. Breach count in the header.
GET	/admin/advisory/:id	advisory.read	-	Media gallery, description, crop, masked caller ID.
POST	/admin/advisory/:id/claim	advisory.claim	yes	Atomic. Second claimer gets 409 ALREADY_CLAIMED.
POST	/admin/advisory/:id/notes	advisory.note.write	-	Outcome, diagnosis, recommendation, call duration.
POST	/admin/advisory/:id/reassign	advisory.reassign	yes	Moves the claim. Keeps the original clock.
POST	/admin/advisory/:id/sla	advisory.sla.override	yes	Step-up. Reason ≥ 20 chars. Logged.
GET	/admin/advisory/shifts	advisory.read	-	Who is on duty across the 08:00-20:00 window.

4.3.10 Platform — fifteen routes

Customers, staff, roles, settings, audit, the PII disclosure register, exports and the dashboard. GET /admin/customers/:id returns a phone masked to the last four and addresses masked to locality. There is no query parameter, no header, no role and no permission that changes that, including SUPER_ADMIN, and the masking happens in a Prisma client extension so a new handler cannot forget it (AA-03 section 3.8). An export of the same rows is masked identically, which is the whole point: the CSV is the obvious way around a masked UI and it is closed.

GET /admin/pii-disclosures is the read side of the one exception in the programme. Every row in it came from a manifest call in 4.6.2, and if that table is ever empty while jobs are being delivered, the disclosure write has broken and the deployment should be rolled back.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/admin/customers	customer.read	-	Masked phone, order count, wallet balance, flags.
GET	/admin/customers/:id	customer.read	-	Profile, addresses masked to locality, order history.
POST	/admin/customers/:id/status	customer.status.write	yes	Step-up. SUSPEND RESTORE + reason.
GET	/admin/staff	staff.read	-	With roles, effective permission count, last login.
POST	/admin/staff	staff.manage	yes	Step-up. Sends an invite; TOTP enrolment is forced.
PUT	/admin/staff/:id/roles	staff.manage	yes	Step-up. Diff shown before submit.
GET	/admin/roles	staff.read	-	System and custom roles with their permission sets.
PUT	/admin/roles/:id/permissions	role.write	yes	Step-up. ALLOW and DENY entries.
GET	/admin/settings	settings.write	-	Typed settings with their current and default values.
PUT	/admin/settings/:key	settings.write	yes	Step-up. Zod-validated per key.
GET	/admin/audit	audit.read	-	Filter by actor, target, permission, date range.
GET	/admin/pii-disclosures	audit.read	-	Who saw which field, when, and why.
POST	/admin/exports	export.create	yes	Queues a masked CSV. Returns a job id.
GET	/admin/exports/:id	export.create	-	Status and, when ready, a 15-minute signed URL.
GET	/admin/dashboard	order.read	-	The eleven tiles. One query, 30 s edge cache.

4.4 PARTNER NAMESPACE ENDPOINTS

34 routes in six groups under /api/v1/partner

Read every path in this section as if it began with "for my org". Not one of the 34 paths contains an org identifier, and that is structural rather than stylistic: `partner_org_id` comes from the verified token claim, is injected by `scoped(prisma, principal)` before the handler sees the query, and a request that carries it in a path, a query string or a body is rejected with 400 `SCOPE_IN_REQUEST` (I-03, AA-02 section 2.4). The rejected alternative was `/partner/:orgId/products` with a check that the claim matches. It lost because the check is a line a developer can forget, and forgetting it in one handler out of thirty-four is a cross-tenant read.

The permission prefix on this namespace is `p.` and the four roles are in `ROLES_PARTNER`. A partner user never sees an admin permission string and never sees another org's row, so there is no tenancy filter in the UI and no org switcher: a person who works for two supplier orgs holds two accounts and logs in twice.

The consumer of every path below is the supplier portal specified in AA-06, which is 26 screens against 34 routes. That ratio is deliberate: several screens are two calls to the same group and no screen calls a route from another namespace, so if a supplier feature appears to need an `/admin` path the feature is wrong, not the contract.

4.4.1 Session — five routes

Login scoped by org slug, conditional TOTP, rotating refresh, invite acceptance and identity. The invite token is single-use with a 72-hour life, and there is no self-registration: a partner org exists because an admin invited it through `/admin/partners`.

METHOD	PATH	PERMISSION	IDEM	NOTE
POST	<code>/partner/auth/login</code>	public	-	Email + password scoped by the org slug in the body.
POST	<code>/partner/auth/totp</code>	challenge token	-	Only when the user has enrolled or holds a finance role.
POST	<code>/partner/auth/refresh</code>	refresh cookie	-	Rotating family, same discipline as staff.
POST	<code>/partner/auth/accept-invite</code>	invite token	-	Sets the first password. Token is single use, 72 h.
GET	<code>/partner/me</code>	session	-	User, org, role, permissions, org status, KYC completeness.

4.4.2 Catalog — eight routes

The supplier's own product authoring. PATCH on an `APPROVED` product does not edit the live row: it forks a new draft revision that re-enters moderation, and the customer keeps seeing the approved version until the new one is approved. A price change is the same shape through `/price-request`. POST `/partner/media/sign` returns a Cloudinary upload signature so the browser uploads directly; the API secret stays server-side and never appears in a `VITE_` variable.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	<code>/partner/products</code>	<code>p.catalog.read</code>	-	Own products only. Scope from the token.
POST	<code>/partner/products</code>	<code>p.catalog.draft</code>	yes	Creates a DRAFT. Not visible to customers.
PATCH	<code>/partner/products/:id</code>	<code>p.catalog.draft</code>	-	Editing an <code>APPROVED</code> product forks a new draft revision.
POST	<code>/partner/products/:id/submit</code>	<code>p.catalog.draft</code>	yes	DRAFT to <code>IN_REVIEW</code> . Enters the admin queue.
POST	<code>/partner/media/sign</code>	<code>p.catalog.draft</code>	-	Returns a Cloudinary signature. The secret stays server-side.
PATCH	<code>/partner/variants/:id/stock</code>	<code>p.catalog.stock</code>	yes	Absolute quantity, not a delta, with an ETag guard.
POST	<code>/partner/variants/:id/price-request</code>	<code>p.catalog.price</code>	yes	Re-enters moderation.
GET	<code>/partner/moderation/history</code>	<code>p.catalog.read</code>	-	Every decision with the reviewer note.

4.4.3 Fulfilment — eight routes

Accept, pack, ready, short-ship, handover and the label. POST /partner/shipments/:id/ready is the point of no return: it broadcasts the delivery offer to wave 1, starts a 600-second TTL, and cannot be undone from this namespace. A supplier who marks ready and then discovers a short-ship has to use /short-ship, which refunds the missing lines automatically rather than cancelling the shipment, because a rider may already be en route.

Nothing in this group ever returns a customer phone number, and the drop is masked to locality and pincode on both the list and the detail. The A4 label from /label carries the shipment code and the drop pincode only. A supplier packing a box does not need the address; the rider does, and that release is 4.6.2 and nowhere else.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/partner/shipments	p.shipment.read	-	Default filter AWAITING_ACCEPTANCE. Masked drop locality only.
GET	/partner/shipments/:id	p.shipment.read	-	Items, qty, weight, COD flag, promised_by. No customer phone.
POST	/partner/shipments/:id/accept	p.shipment.accept	yes	Body carries ready_by. Rejects a time past the SLA.
POST	/partner/shipments/:id/pack	p.shipment.pack	yes	ACCEPTED to PACKING.
POST	/partner/shipments/:id/ready	p.shipment.pack	yes	Triggers the offer broadcast. Point of no return.
POST	/partner/shipments/:id/short-ship	p.shipment.shortship	yes	Per-item shortfall + reason. Auto-refunds.
POST	/partner/shipments/:id/handover	p.shipment.pack	yes	Confirms the rider scan from the supplier side.
GET	/partner/shipments/:id/label	p.manifest.print	-	A4 PDF: pick list, shipment code, drop pincode.

4.4.4 Returns — three routes

Inbound returns and RTO. /receive records condition as SELLABLE, DAMAGED or MISSING, and /dispute is open only for 48 hours after receive. The window exists because a dispute filed three weeks later cannot be adjudicated: the photo evidence is stale and the rider who carried the box has done 200 jobs since.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/partner/returns	p.return.read	-	Returns and RTO inbound, with the customer photo evidence.
POST	/partner/returns/:id/receive	p.return.receive	yes	Confirms condition: SELLABLE DAMAGED MISSING.
POST	/partner/returns/:id/dispute	p.return.dispute	yes	Only within 48 h of receive. Goes to an ops queue.

4.4.5 Finance — five routes

Payout visibility, the ledger, the self-billed commission invoice and bank details. PUT /partner/bank is step-up guarded and has a side effect that surprises people: the account enters PENDING_VERIFICATION and payouts pause until an admin verifies it through /admin/partner-bank/:id/verify. That pause is deliberate. Bank-detail replacement is the classic payout-diversion attack, and a paused payout is recoverable where a released one is not. Every amount in this group is integer paise and the UTR is masked to the last four.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/partner/payouts	p.payout.read	-	Accrued, in-run, paid and withheld lines.
GET	/partner/payout-runs/:id	p.payout.read	-	Line detail, commission, TDS, net, UTR last four.
GET	/partner/ledger	p.payout.read	-	Append-only running balance.
GET	/partner/invoices/:runId	p.invoice.download	-	Self-billed commission invoice as PDF.
PUT	/partner/bank	p.bank.manage	yes	Step-up. Enters PENDING_VERIFICATION; payouts pause.

4.4.6 Org — five routes

Users, invites and the KYC checklist. /partner/users/:id/disable refuses to remove the last PARTNER_OWNER with 409 LAST_OWNER, because an org with no owner cannot invite anyone and becomes an admin support ticket. Re-uploading a KYC document resets it to SUBMITTED and it re-enters the admin review queue.

Both write routes here are step-up guarded, which is unusual for a namespace where TOTP is conditional. The resolution is that enrolling a factor is a precondition of holding `p.user.manage` at all: a role that can invite a colleague can invite themselves a second seat with a finance role, so user management is a privilege-escalation surface and is treated as one. A partner user without an enrolled factor who holds the permission gets 401 `STEP_UP_REQUIRED` with an enrolment link rather than a challenge, which is the only place in the contract where that response carries a URL.

Invites are email-only and single-use, and the role code is fixed at invite time. There is no route that changes a partner user's role: the org disables the seat and invites a new one. That is deliberately clumsy for a four-role namespace, and it means one fewer write path against the table that decides who can see a bank account.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/partner/users	p.user.manage	-	Org users with roles and last login.
POST	/partner/users	p.user.manage	yes	Step-up. Invite by email with a role code.
POST	/partner/users/:id/disable	p.user.manage	yes	Step-up. Cannot disable the last <code>PARTNER_OWNER</code> .
GET	/partner/kyc	p.kyc.submit	-	Checklist with per-document status and reviewer notes.
POST	/partner/kyc	p.kyc.submit	yes	Uploads or replaces one document. Resets it to <code>SUBMITTED</code> .

4.5 FLEET NAMESPACE ENDPOINTS

37 routes in six groups under `/api/v1/fleet`

The same scoping rule as 4.4 applies without exception: none of the 37 paths carries a `fleet_org_id`, the claim is the only source of tenancy, and a request that supplies it gets 400 `SCOPE_IN_REQUEST`. There is one extra layer here that the partner namespace does not have. A `FLEET_RIDER` is scoped a second time, to their own rider id, inside the org scope: `GET /fleet/jobs` returns the org's jobs for a dispatcher and only the rider's own jobs for a rider, and that narrowing is in the handler rather than in the permission, because it is a row filter and not a capability.

This is also the namespace to be most careful with. It is the only one a hostile party can plausibly get an account on: a delivery company signs up, passes a document check, and is inside. That is why the fleet rate-limit bucket is tighter than admin (4.8), why the manifest disclosure is job-scoped and window-scoped (4.6.2), and why the accept race is settled in the database rather than in application code (4.6.1).

AA-07 specifies the 24 screens that consume these 37 routes, and two of them are rider-only and mobile-only. Read the two together when estimating: a dispatcher screen and a rider screen can call the same path and get a different row set, and that difference is invisible in the table below because it is a handler-level row filter rather than a permission.

4.5.1 Session — five routes

Password login for office roles, mobile-OTP login for riders, conditional TOTP for owner and finance, rotating refresh, identity. The OTP request route is idempotency-keyed so a rider double-tapping does not burn two codes; the code itself is a bcrypt hash in the row and appears in no log line.

METHOD	PATH	PERMISSION	IDEM	NOTE
POST	/fleet/auth/login	public	-	Email + password, or mobile OTP for a <code>FLEET_RIDER</code> .
POST	/fleet/auth/otp/request	public	yes	Rider login. Same WhatsApp-first channel as the app.
POST	/fleet/auth/totp	challenge token	-	Owner and finance roles only.
POST	/fleet/auth/refresh	refresh cookie	-	Rotating family.
GET	/fleet/me	session	-	User, org, role, tier, COD exposure, blocked flag.

4.5.2 Offers — four routes

The auction surface. An offer shows the drop pincode, the weight band, the COD amount and the fee, and never the address or the customer. Accept is the one genuinely concurrent write in the programme and is dissected in

4.6.1. GET /fleet/offers/stream is the only server-sent-events endpoint anywhere in this contract, it degrades to 10-second polling when the connection drops, and it is cut candidate C-1: if Cloud Run's connection accounting makes it expensive the stream goes and the polling stays.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/fleet/offers	f.offer.read	-	Live offers with seconds remaining. Drop pincode only, no address.
POST	/fleet/offers/:id/accept	f.offer.respond	yes	First accept wins. Losers get 409 OFFER_ALREADY_TAKEN.
POST	/fleet/offers/:id/decline	f.offer.respond	yes	Reason code. Declines feed the acceptance-rate metric.
GET	/fleet/offers/stream	f.offer.read	-	Server-sent events. Falls back to 10 s polling.

4.5.3 Jobs — ten routes

The whole delivery leg: assignment, four scans, POD, failure and RTO. Order matters and the FSM enforces it, so /out-for-delivery on a job still in PICKED_UP is 409 rather than a silent jump. /out-for-delivery is also what opens the manifest window, so a rider who scans it in the warehouse at 9 a.m. and delivers at 4 p.m. will find the manifest closed: 240 minutes is measured from the scan, not from the delivery attempt.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/fleet/jobs	f.job.read	-	Own jobs. A FLEET_RIDER sees only jobs assigned to that rider.
GET	/fleet/jobs/:id	f.job.read	-	Masked view: pincode, locality, weight, COD amount, promised_by.
GET	/fleet/jobs/:id/manifest	f.job.manifest	-	THE disclosure endpoint. Full address. Writes pii_disclosures.
POST	/fleet/jobs/:id/assign	f.job.assign	yes	Attach rider + vehicle. Checks the 8-job rider cap.
POST	/fleet/jobs/:id/pickup	f.job.scan	yes	Scan the shipment code at the supplier gate.
POST	/fleet/jobs/:id/transit	f.job.scan	yes	PICKED_UP to IN_TRANSIT.
POST	/fleet/jobs/:id/out-for-delivery	f.job.scan	yes	Opens the manifest window.
POST	/fleet/jobs/:id/pod	f.pod.write	yes	OTP + geo + optional photo. Validates the 200 m fence.
POST	/fleet/jobs/:id/fail	f.job.fail	yes	Outcome code + geo. Third failure forces RTO.
POST	/fleet/jobs/:id/rto	f.job.scan	yes	Starts or completes the reverse leg.

4.5.4 Fleet — eight routes

Riders, vehicles and service areas. A new rider enters PENDING_DOCS and cannot be assigned a job until documents are uploaded, which is the fleet's own control rather than ours. Vehicles matter more than they look: adding a PICKUP or TRUCK unlocks heavy-equipment offers, so the vehicle list is an eligibility declaration and a fleet that adds a truck it does not own will be offered work it cannot do and lose acceptance rate. Registration numbers are stored and displayed as last four only.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	/fleet/riders	f.rider.manage	-	Riders with status, live job count and rating.
POST	/fleet/riders	f.rider.manage	yes	Name, mobile, licence. Enters PENDING_DOCS.
PATCH	/fleet/riders/:id	f.rider.manage	-	Status, leave, vehicle default.
POST	/fleet/riders/:id/documents	f.rider.manage	yes	Signed Cloudinary upload for licence and ID.
GET	/fleet/vehicles	f.vehicle.manage	-	Registration last four, kind, capacity, insurance expiry.
POST	/fleet/vehicles	f.vehicle.manage	yes	Adding a PICKUP or TRUCK unlocks heavy-equipment offers.
GET	/fleet/areas	f.area.request	-	Current approved pincodes and rate card.
POST	/fleet/areas/request	f.area.request	yes	Bulk pincode add or remove, plus fee changes. Needs approval.

4.5.5 Finance — six routes

Earnings, the ledger, COD declaration and bank details. COD is the sharp edge of the whole programme: cash a rider collects is our money in the fleet's hands, so `/fleet/cod/:id/declare` is step-up guarded and takes a declared amount plus a bank reference, and the admin side reconciles it against expected cash in `/admin/cod/:id/reconcile`. A variance beyond `COD_VARIANCE_BLOCK_BPS`, currently 200 basis points, blocks the fleet from further COD work and every subsequent accept on a COD job returns `409 FLEET_BLOCKED_COD`.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	<code>/fleet/earnings</code>	<code>f.payout.read</code>	-	Per-job lines with base, distance, COD fee and penalties.
GET	<code>/fleet/payout-runs/:id</code>	<code>f.payout.read</code>	-	Run detail with UTR last four.
GET	<code>/fleet/ledger</code>	<code>f.payout.read</code>	-	Append-only balance including COD held.
GET	<code>/fleet/cod</code>	<code>f.cod.declare</code>	-	Open periods with expected cash and age.
POST	<code>/fleet/cod/:id/declare</code>	<code>f.cod.declare</code>	yes	Step-up. Declared amount + bank reference.
PUT	<code>/fleet/bank</code>	<code>f.bank.manage</code>	yes	Step-up. Pauses payouts until admin verifies.

4.5.6 Org — four routes

Users and company KYC. Two of these four declare only session rather than a permission string, because KYC completion is the gate that lets an org do anything at all and gating it behind a permission the owner cannot yet grant themselves is a deadlock.

Fleet KYC is heavier than partner KYC and that is why it has its own checklist: a transport licence, vehicle insurance with an expiry we track, and identity documents for the office seats. Insurance expiry is the one field an admin will chase, because an expired policy on a vehicle carrying our COD cash is a liability nobody has budgeted for. The contract does not block offers on an expired policy today; it surfaces the date in `/admin/fleets/:id` and leaves the decision to a human, which is a gap worth naming rather than pretending the check exists.

POST `/fleet/users` is step-up guarded on the same escalation reasoning as 4.4.6. Rider seats are created here too, and a rider seat is the cheapest credential in the programme: mobile plus OTP, no password, no factor. That is why a rider is row-scoped to their own jobs inside the org scope and why nothing a rider can call returns another rider's work.

METHOD	PATH	PERMISSION	IDEM	NOTE
GET	<code>/fleet/users</code>	<code>f.user.manage</code>	-	Org users with roles.
POST	<code>/fleet/users</code>	<code>f.user.manage</code>	yes	Step-up. Invite dispatch, finance or rider seats.
GET	<code>/fleet/kyc</code>	<code>session</code>	-	Company KYC checklist including transport licence and insurance.
POST	<code>/fleet/kyc</code>	<code>session</code>	yes	Upload or replace one document.

4.6 THE FOUR ENDPOINTS THAT CARRY REAL RISK

Full request and response shapes, and every way each one fails

The other 160 routes are CRUD with a permission on the front. These four are not. Each one is either genuinely concurrent, or releases personal data, or moves cash, and each one has a failure mode that costs somebody real money if it is implemented from the note field alone. They are written out here in full because they are the four places where a plausible-looking implementation is wrong.

4.6.1 POST `/fleet/offers/:id/accept` — the race

When a shipment is marked ready, the offer goes to every eligible fleet in wave 1 at the same instant, with a 600-second TTL. Ten dispatchers see it at once and two or three tap accept within the same second. Exactly one must win, the losers must be told cleanly, and no job may end up attached to two fleets. This is settled by one conditional UPDATE and its affected-row count, inside the request transaction.

A read-then-write in application code loses here: SELECT status FROM offers, decide, UPDATE. Two requests both read AVAILABLE and both write ACCEPTED, and the second overwrites the first with no error anywhere. Advisory locks would also work and were rejected as more machinery than a WHERE clause. The row count is the only thing the handler branches on, and the handler never trusts a value it read earlier in the same request.

```
POST /api/v1/fleet/offers/9f21c8/accept      Authorization: Bearer aud=atv_fleet
Idempotency-Key: 3b91-77ac-...              { "vehicle_kind": "BIKE" }

UPDATE offers SET status = 'ACCEPTED', accepted_by_fleet_id = $claim, accepted_at = now()
WHERE id = $1 AND status = 'AVAILABLE' AND expires_at > now();      -- 1 row => winner

200 { "data": { "job_id": "7c40ab", "shipment_code": "ATV-SHP-40912",
               "drop_pincode": "751024", "cod_amount_paise": 148000,
               "promised_by": "2026-08-30T18:30:00+05:30", "fee_paise": 6500 },
      "cursor": null }
```

CODE	HTTP	WHEN IT HAPPENS	WHAT THE CLIENT DOES
OFFER_ALREADY_TAKEN	409	The UPDATE affected zero rows and the offer row is ACCEPTED. Another fleet won by microseconds.	Remove the card, do not retry, refresh the offer list. A retry can never succeed.
OFFER_EXPIRED	409	Zero rows and expires_at is in the past. The wave TTL (600, 600, 900 seconds by wave) elapsed.	Remove the card. The job has gone to the next wave or to the exception board.
FLEET_BLOCKED_COD	409	Checked before the UPDATE. COD variance for this fleet exceeds COD_VARIANCE_BLOCK_BPS = 200.	Show the block reason and the open COD balance, and link to the declare screen.
VALIDATION_FAILED	400	No vehicle of the declared kind is registered, or the rider cap of 8 active jobs is already met.	Show which precondition failed. This one is fixable and a retry may succeed.
IDEMPOTENCY_MISMATCH	409	Same key replayed with a different body, usually a client bug reusing a key across two offers.	Treat as a bug, log it, generate a fresh key per user action.

ONE NAMING WART, STATED SO NOBODY CHASES IT

The note field on this row in the endpoint table says "Losers get 409 OFFER_TAKEN". The canonical code in packages/shared/errors.ts and in AA-02 section 2.9 is OFFER_ALREADY_TAKEN. They are the same failure; the short form is shorthand in the note and is not a second code. The fact-base note should be corrected before freeze.

4.6.2 GET /fleet/jobs/:id/manifest — the only PII disclosure in the programme

Everywhere else in this contract, and in all three portals, personal data is last four characters only, with no reveal affordance and no permission that unmask it, not even SUPER_ADMIN. This endpoint is the single exception, and it exists because a rider standing on a road in Khandagiri cannot deliver to a locality. It releases the delivery address and nothing else. The customer phone number is absent from the response under every code path; there is no field for it, no flag that adds one, and the masked_phone in the job detail stays masked. A rider who needs to call goes through the masked-calling gap named in AA-00 section 0.9, which is a paid feature we have not bought.

Three conditions gate the release and all three are checked in the handler before anything is read. The job must belong to the caller's fleet_org_id, and for a rider, to that rider. The job must be in OUT_FOR_DELIVERY. And the call must be within MANIFEST_WINDOW_MINUTES = 240 of the out-for-delivery scan. If all three hold, unmaskAddress() decrypts and a pii_disclosures row is inserted in the same transaction: same commit, no queue, no after-hook. If the insert fails the address is not returned, because an undisclosed disclosure is worse than a failed delivery attempt.

```
GET /api/v1/fleet/jobs/7c40ab/manifest      permission f.job.manifest

200 { "data": { "job_id": "7c40ab", "consignee_name": "S. Pradhan",
  "address_line1": "Plot 214, Sector A", "address_line2": "Near Ganesh Temple",
  "locality": "Khandagiri", "city": "Bhubaneswar", "pincode": "751030",
  "landmark": "blue gate", "cod_amount_paise": 148000,
  "window_expires_at": "2026-08-30T20:14:00+05:30" }, "cursor": null }

-- same transaction, always:
INSERT INTO pii_disclosures (actor_kind, actor_id, subject_user_id, field, reason, job_id,
  disclosed_at) VALUES ('fleet', $1, $2, 'address', 'DELIVERY', $3, now());

NOT PRESENT IN ANY RESPONSE: phone, alternate phone, email, full payment reference.
```

CODE	HTTP	WHEN IT HAPPENS	WHAT THE CLIENT DOES
DISCLOSURE_WINDOW_CLOSED	403	More than 240 minutes since the out-for-delivery scan, or the job has since reached DELIVERED, FAILED or RTO.	Show "window closed, ask dispatch to reopen". Reopening means a new out-for-delivery scan.
VALIDATION_FAILED	400	The job is not in OUT_FOR_DELIVERY. Common when the rider skipped the scan and went straight to the door.	Prompt the rider to scan out-for-delivery first. The scan is the consent record.
NOT_FOUND	404	The job exists but belongs to another fleet org, or to another rider. Deliberately not 403.	Nothing. A 403 here would confirm the job id exists, which is a cross-tenant leak of a different kind.
PERMISSION_DENIED	403	The caller's role does not hold f.job.manifest. A finance seat never holds it.	Hide the manifest button for roles without the permission.

4.6.3 POST /fleet/jobs/:id/pod — proof of delivery

The delivery OTP is a four-digit code the customer receives on WhatsApp when the job goes out for delivery. It is stored as a bcrypt hash on the job row and never as digits: not in the column, not in a log line, not in a Sentry breadcrumb, and it is never returned by any endpoint in any namespace. The rider types what the customer reads out and the handler compares against the hash. Everything else in the request is evidence rather than authentication: the geo point is validated against `POD_GEOFENCE_METRES = 200` from the address geocode, the photo is a Cloudinary public id from a signed upload, and for a COD job the declared collected amount is mandatory and must equal `cod_amount_paise` exactly.

A mismatch increments `pod_otp_attempts` on the job and after three the OTP is locked and the rider must use `/fleet/jobs/:id/fail` with outcome `CUSTOMER_UNREACHABLE`. The lockout is harsher than the login lockout in 4.2.3 on purpose: a locked login costs a rider fifteen minutes, while an unlimited POD OTP lets a rider brute-force 10,000 codes and mark a parcel delivered that they kept. Out-of-fence deliveries are not blocked, because rural geocodes are wrong often enough that blocking would strand real deliveries; they are recorded and flagged for review through `/admin/pod/:id/flag`.

```
POST /api/v1/fleet/jobs/7c40ab/pod      Idempotency-Key: cld4-...
{ "otp": "4192", "lat": 20.2591, "lng": 85.7758, "accuracy_m": 12,
  "photo_public_id": "atv/pod/7c40ab_1", "cod_collected_paise": 148000 }

200 { "data": { "job_status": "DELIVERED", "shipment_status": "DELIVERED",
  "order_status": "DELIVERED", -- derived, never written
  "distance_from_drop_m": 43, "geofence_ok": true,
  "cod_ledger_entry_id": "le_88f102", "pod_id": "pod_5512" }, "cursor": null }
```

CODE	HTTP	WHEN IT HAPPENS	WHAT THE CLIENT DOES
POD_OTP_MISMATCH	422	bcrypt compare failed. Attempt counter incremented; the response carries <code>attempts_remaining</code> .	Show attempts remaining. After the third, switch the screen to the failure flow.
POD_OTP_LOCKED	423	Three failed attempts on this job. The OTP cannot be retried and is not reissued.	Force <code>/fail</code> with an outcome code. Dispatch can reschedule.
OUT_OF_GEOFENCE	422	Only when the fleet is under a POD review hold. Otherwise the distance is recorded and the delivery proceeds.	Ask for a photo and retry. The distance is in the response either way.
COD_AMOUNT_MISMATCH	422	<code>cod_collected_paise</code> does not equal <code>cod_amount_paise</code> . Partial collection is not a delivery.	Do not allow an edited amount. Use <code>/fail</code> with <code>CUSTOMER_REFUSED_PAYMENT</code> .

4.6.4 POST /admin/payout-runs — the settlement run

A payout run assembles every accrued line for one kind of counterparty and one period into a DRAFT, which lock freezes into a bank file and release turns into money that has left the account. The identity of a run is (kind, period_start, period_end), and the Idempotency-Key for the create call is derived from exactly that, not generated per click. A second POST with the same run key returns the existing run with Idempotency-Replay: true and creates nothing. That is the whole defence against the most expensive bug this programme could ship: paying every supplier twice because a finance operator refreshed a slow page.

The create call accepts dry_run: true, which assembles the run in a transaction, returns the totals and line count, and rolls back. Finance uses it to reconcile against their own expectation before committing, and a dry run consumes no idempotency key because it writes nothing. Withholding a line before lock is the only way to remove a counterparty from a run; after lock the membership is frozen and a bad line is handled by a compensating entry in the next period, never by editing the locked run.

```
POST /api/v1/admin/payout-runs      permission payout.run.create
Idempotency-Key: run:PARTNER:2026-08-16:2026-08-31      -- derived from the run key, not random
{ "kind": "PARTNER", "period_start": "2026-08-16", "period_end": "2026-08-31", "dry_run": false }

201 { "data": { "run_id": "pr_2026H2_08", "status": "DRAFT", "line_count": 41,
               "gross_paise": 918445000, "commission_paise": 64291150,
               "tds_paise": 9184450, "net_paise": 844969400 }, "cursor": null }

Replay of the same key -> 200 + Idempotent-Replay: true, byte-identical body, no second run.
```

CODE	HTTP	WHEN IT HAPPENS	WHAT THE CLIENT DOES
IDEMPOTENCY_MISMATCH	409	Same run key, different period or kind in the body. Almost always a copy-pasted key.	Stop. Do not retry with a new key until the period is checked against the existing run.
VALIDATION_FAILED	400	A run already exists for this kind and period in DRAFT or LOCKED, or the period is still open.	Link to the existing run. Two live runs for one period is the state we refuse to represent.
STEP_UP_REQUIRED	401	Only on /release, not on create or lock. Fresh factor older than 300 seconds.	Open StepUpDialog, then replay /release with the same key.
LEDGER_IMMUTABLE	409	Something attempted an UPDATE on a payout ledger row. Raised by the Postgres rule, not by the handler.	Treat as a bug and stop the run. Corrections are compensating rows only.

4.7 WEBHOOKS AND INBOUND INTEGRATIONS

Three inbound callbacks, and the rule that none of them writes a ledger

Inbound callbacks live under /api/v1/hooks and are outside the middleware chain of AA-02 section 2.3: there is no JWT, no actor, no tenant claim and no permission, because the caller is Razorpay or Meta and has none of those. Authentication is signature verification instead, and the route is otherwise treated as hostile input. The body is read as a raw buffer before any JSON parsing, because HMAC over a re-serialised object is a signature check that passes when it should fail.

The rule that matters more than the signature: a webhook never writes a ledger row, never transitions an FSM and never touches money. It verifies, deduplicates on the provider event id, inserts a row into the job queue, and returns 200. All state change happens in the worker, inside a transaction, under the same invariants as any other write. The reason is retries: Razorpay will redeliver the same event, and a handler that credits a payment inline turns a redelivery into a double credit. A queue row with a unique constraint on the provider event id turns it into a no-op.

ENDPOINT	SOURCE	VERIFICATION	WHAT IT ENQUEUES
POST /hooks/razorpay/payment	Razorpay	HMAC-SHA256 over the raw body with RAZORPAY_WEBHOOK_SECRET, constant-time compare, 300-second timestamp window, dedupe on event id	reconcile-payment: match to the order, insert the payment ledger entry, recompute the derived order status
POST /hooks/razorpay/refund	Razorpay	Same secret and same window. A refund event for an unknown reference is stored and alerted, never guessed at	settle-refund: close the refund row, insert the compensating ledger entry, notify the customer

ENDPOINT	SOURCE	VERIFICATION	WHAT IT ENQUEUES
POST /hooks/whatsapp/status	Meta Cloud API	Verify token on the GET handshake, then app-secret HMAC on every POST	notification-status: mark the template message sent, delivered, read or failed for the retry policy

Two things are deliberately not webhook-driven. Razorpay settlement reports are polled daily by a cron, because the settlement webhook is not available on the plan we are on and paying for it is not in a Rs 0 budget; the cron reads the report API and the reconciliation is therefore up to 24 hours behind, which finance accepts. Delivery progress is also not pushed: the fleet portal and the admin exception board poll, because the only push surface we have is the fleet offer stream and extending it to every board would mean holding thousands of connections on a service that bills by request-seconds.

4.8 RATE LIMITS AND QUOTAS

Five buckets, one Postgres table, and why admin is the loosest

The limiter is the fifth step of the middleware chain and runs before JWT verification, so an unauthenticated flood costs one indexed insert rather than an RS256 verify. A bucket is a row keyed by (bucket key, minute) with a counter, incremented with an upsert. There is no Redis: a managed Redis instance is the cheapest paid line item that would break I-14, and at our request volumes a Postgres upsert against a Neon instance we are already paying nothing for is fast enough. The tradeoff is honest — the limit is per-minute and not a sliding window, so a caller can spend a whole minute's allowance in the first two seconds twice across a minute boundary.

NAMESPACE	BUCKET KEY	LIMIT	BURST	WHY THIS NUMBER
/api/v1/public	client IP	30 / min	10	Unauthenticated. Catalogue browsing only. An IP behind a village Wi-Fi router is the reason it is not lower.
/api/v1/app	user id	120 / min	30	One human with a phone. Generous enough for a fast scroll, tight enough to notice a scripted client.
/api/v1/admin	staff id	240 / min	60	The loosest, on purpose. A dispatcher with four list views open on 20-second polling is a legitimate heavy caller.
/api/v1/partner	partner_org_id	120 / min	30	Per org, not per user, so a supplier with six packing terminals shares one bucket.
/api/v1/fleet	fleet_org_id	180 / min	45	Higher than partner because riders poll offers; lower than admin because this is the namespace an outsider can register for.

Exceeding the bucket is 429 RATE_LIMITED with Retry-After set to the seconds remaining in the minute. The generated client honours it with a single scheduled retry and no exponential backoff, because backoff on a per-minute window just means waiting longer than necessary. Two quotas sit outside the limiter and are enforced in handlers: EXPORT_ROW_CAP = 5000 rows per CSV export, which turns a request for the full order history into four exports, and the 8-job cap per rider from RIDER_MAX_ACTIVE_JOBS. Both return 400 VALIDATION_FAILED rather than 429, because they are not about rate.

4.9 VERSIONING AND DEPRECATION

Additive-only inside a major, and what that costs

There is one API version, v1, and the intention is that there is never a v2 during this programme. The reason is the mobile app: an installed Android build cannot be forced to update, so a removed field is a crash on a phone belonging to a farmer who will not be told why. Inside v1 every change must be additive, and the ESLint rule plus the contract sweep in AA-01 section 1.7 fail the build on a removed or retyped field in @atavishaala/contract. The portals are different: they are served from Cloudflare Pages and every reload is the current build, so a portal-only field can move faster. It still goes through the same contract package, because two sets of rules for one schema is how a mobile field gets deleted by someone who thought it was portal-only.

CHANGE	ALLOWED INSIDE v1	HOW IT IS DONE
Add a response field	Yes	Add to the Zod schema as optional, regenerate, bump the minor version of @atavishaala/contract. Old clients ignore it.
Add a request field	Yes, if optional with a default	A required new field is a breaking change even when every current caller would send it. Make it optional, backfill, then tighten in the next major.

CHANGE	ALLOWED INSIDE v1	HOW IT IS DONE
Retire a field	Yes, in three steps	Announce in the Monday sync and mark @deprecated in the schema; then keep returning it with a null or a frozen value; remove only in a major. Never remove and never retype inside v1.
Change a status enum	Only by adding a member	Clients must treat an unknown enum member as "unknown" rather than crashing, and the generated TypeScript union is widened with a catch-all for that reason.
Change a path or method	No	A new path beside the old one, with the old one marked deprecated. Two live paths for one operation until the next major.

A deprecated route answers normally and adds two headers: Deprecation with the date it was marked, and Sunset with the earliest date it may be removed, which is never inside the current major. Both are logged with the caller's actor id, so before anything is removed there is a list of exactly who is still calling it rather than a guess.

4.10 WHAT THIS CONTRACT DOES NOT COVER

Six gaps, each with the workaround and what closing it would cost

These are not oversights and they are not on a roadmap slide. Each one was a decision with a named alternative, and each one has a workaround that somebody has to live with. Anyone estimating work against this contract should read this section before the endpoint tables, because four of the six show up as "why is this screen slow" or "why is this ten calls" during week three.

GAP	CONSEQUENCE TODAY	WORKAROUND, AND THE COST OF CLOSING IT
No GraphQL and no field selection	Some admin list endpoints over-fetch. GET /admin/orders returns the full item array whether the board needs it or not.	Purpose-built list shapes for the three boards ops uses most, and nothing for the rest. GraphQL was rejected on cost of a second contract surface plus per-field authorisation, which is where the PII rule would leak.
No server-sent events or websockets outside the fleet offer stream	The admin exception board polls every 20 seconds. That timer is the largest single source of Cloud Run requests in the programme.	Nothing to do about it inside Rs 0: held connections bill by request-second on Cloud Run and the free tier is measured in exactly that. If the request count becomes the binding cost, the poll interval goes to 45 seconds before anything else changes.
No bulk write endpoints	A 200-row catalogue update is 200 calls unless the CSV path is used, and 200 calls at 240 per minute is a minute of a partner staring at a progress bar.	The CSV importer covers the real case and runs as a queued job. A generic bulk endpoint was rejected because partial failure inside one request has no honest response shape.
No public partner API	A supplier with their own ERP cannot integrate. They use the portal, or they send a CSV.	Out of scope in this programme. It needs API keys, per-key rate limits, a sandbox and a support commitment, which is a product, not an endpoint.
No customer phone anywhere, including for riders	A rider who cannot find a house cannot call. They call dispatch, who call the customer from a personal handset.	Masked calling from Exotel or Twilio, roughly Rs 0.60 per minute plus a monthly number rental. Named as a paid gap in AA-00 section 0.9 and not bought.
No OpenAPI-driven mock server	Front-end pairs code against a generated client with hand-written fixtures, so a shape mismatch is found at integration rather than at build.	The contract sweep in AA-01 section 1.7 catches the mismatch within a day. A hosted mock is another always-on service and therefore another recurring cost.

THE ONE THING TO TAKE AWAY FROM THIS DOCUMENT

The tables in 4.3 to 4.5 are generated from the same fact base the rest of the set is generated from, so they cannot disagree with AA-02, AA-03 or AA-08 about a path or a permission. Where this document and a wiki page disagree, this document is right. Where this document and packages/shared/routes.ts disagree, the code is right and the fact base has a bug worth fixing the same day.